RevConnect Project Documentation

Overview

RevConnect is a next-generation social networking ecosystem designed to bridge the gap between personal expression, creative content distribution, and professional business engagement. Built on the robust **Spring Boot 3** framework and leveraging **Java 21**, the platform is engineered for scalability, security, and a highly personalized user experience.

At its core, RevConnect solves the fragmentation issue of modern social media by offering three distinct user paradigms within a single unified platform:

- **Personal Connectivity**: Empowering users to build genuine relationships through high-fidelity connection workflows, real-time messaging, and shared content feeds.
- **Creator Empowerment**: Providing creators with the tools to showcase their work, build a dedicated following, and engage with their audience through specialized category-based discovery.
- **Business Integration**: A comprehensive suite for enterprises to manage professional brand pages, showcase product catalogs, and leverage social commerce through sophisticated product-tagging in content.

The platform is not just a social network; it's a technical demonstration of modern full-stack engineering, implementing dual-layer security, N-layer architecture, and data-driven insights.

Technology Stack

The application is built on a robust enterprise-grade stack:

- **Core Framework**: Spring Boot 3.2.3
- **Language**: Java 21
- **Templating Engine**: Thymeleaf (for server-side HTML rendering)
- **Security**: Spring Security 6 with JWT (JSON Web Tokens)
- **Data Access**: Spring Data JPA (Hibernate)
- **Database**: H2 (Development/File-based), Oracle (Production-ready)
- **Logging**: Log4j2
- **Build Tool**: Maven

Core Features

1. **Authentication & Security**:
    - Secure login and registration with BCrypt password encoding.
    - Dual-layer security: Session-based for Web UI and Stateless JWT for REST API.
    - Role-based Access Control (RBAC) across all layers.
2. **Social Feed**:

    - Dynamic feed displaying posts from connections and follows.
    - Support for text and image-based content with automated validation.
3. **Interactions**:
    - High-performance Like and Comment systems with unique constraint protections.
    - Reshare/Repost support with reference tracking to original content.
    - Direct messaging with read-status tracking.
4. **Networking**:
    - Connection request flow: Request -> Pending -> Accepted/Rejected.
    - Decentralized follower/following mechanism similar to modern social platforms.
5. **Business & Commerce**:
    - Dedicated Business Pages with custom branding fields (About, Address, Hours).
    - Integrated Product Catalog allowing business users to manage inventory.
    - Product tagging in posts to drive commerce engagement.

Security Implementation

The platform implements a sophisticated security model using **Spring Security 6**:

- **Authentication Strategy**:
    - WebSecurityConfig defines two distinct filter chains.
    - **API Chain**: Uses JwtRequestFilter for stateless authentication on /api/** endpoints.
    - **Web Chain**: Uses standard session-based form login for the Thymeleaf-fronted UI.
- **Identity Management**:
    - CustomUserDetailsService integrates with UserRepository to load user principals.
    - Supports both username and email as valid login identifiers.
- **Encryption**: All passwords are encrypted using BCryptPasswordEncoder before storage.

User Roles & Workflows

The application business logic is specialized based on user roles:

- **PERSONAL**: Standard social features. Can create posts, follow users, and send connection requests.
- **CREATOR**: Enhanced engagement features. Includes category selection and advanced showcases.
- **BUSINESS**: Operations-focused. Access to Business Pages, Product Catalogs, and commerce-oriented analytics.

Data Handling & Quality

- **Mappers & DTOs**: The project strictly separates internal entities from external API contracts using DTOs (Data Transfer Objects) and Mappers. This ensures internal schema details are not exposed.
- **Validation**: Strict use of @Validation annotations for incoming data, including email format checks, size constraints, and nullability.
- **Testing Strategy**:
    - **Unit Testing**: Leveraging **JUnit 5** and **JUnit Vintage** for comprehensive logic testing.
    - **Mocking**: Using **Mockito** to isolate services from data access during testing.
    - **Integration Testing**: Using **H2 In-memory database** to run integration tests without affecting production data. The project follows a classic **N-Layer Architecture**:

1. **Controller Layer**: Handles incoming web and REST requests.
2. **Service Layer**: Implements core business logic and cross-component coordination.
3. **Repository Layer**: Abstracts database operations using standard JPA patterns.
4. **Data Model**: Rich domain model with entities and DTOs for data transfer.

Database Schema

The database is structured to support complex social relationships.

- **Core Entities**: users, posts, business_pages, products.
- **Relationship Entities**: connections, follows, likes, comments.

Project Directory Structure

```
RevConnect_P2/
 ├– src/main/java/com/rev/app/
| ├– config/       # Security, Web, and Application configurations
| ├– controller/   # Thymeleaf-based Web Controllers
| ├– rest/         # RESTful API Controllers (JWT Secured)
| ├– service/      # Business Logic implementation
| ├– repository/   # Spring Data JPA Repositories
| ├– entity/       # Database Entity definitions
| ├– dto/          # Data Transfer Objects
| ├– mapper/       # Class mapping logic
```

```
|  └─ security/     # JWT specific security components
 ├─ src/main/resources/
|  ├─ templates/    # HTML5 / Thymeleaf views
|  ├─ static/       # CSS, JavaScript, and Images
|  └─ application.properties # Main application configuration
 ├─ data/           # Local H2 database storage
└─ pom.xml          # Maven project configuration
```

Getting Started

Running the Application

Ensure you have **JDK 21** and **Maven** installed.

bash

mvn spring-boot:run

Base URL: http://localhost:8083

Accessing the Database

The H2 Console is enabled for development: URL: http://localhost:8083/h2-console Driver Class: org.h2.Driver JDBC URL: jdbc:h2:file:./data/revconnectdb

REST API Reference

The platform provides a growing REST API for external integrations and mobile clients.

- **Auth**: POST /api/auth/authenticate - JWT token generation.
- **Users**:
    - GET /api/users/search - Search for users with summaries.
    - GET /api/users/me - Get current authenticated user details.
- **Posts**:
    - GET /api/posts/feed - Retrieve the personalized social feed.
    - POST /api/posts - Create new content.
    - POST /api/posts/{id}/like - Toggle like status.

Exceptional Handling & Quality Control

- **Global Exception Handler**: Centrally manages errors via GlobalExceptionHandler.
- **Custom Exceptions**:
    - ResourceNotFoundException (404): Triggered when an entity (User/Post) doesn't exist.

    - UserAlreadyExistsException (409): Triggered during duplicate registration attempts.
    - AccessDeniedException (403): Triggered when a user attempts an action outside their role or ownership.
- **Transactional Integrity**: All service-layer methods are marked with @Transactional to ensure data consistency during complex multi-step operations (e.g., following a user and sending a notification).

Trace: Core Data Flow

Example: **Creating a Post**

1. **Controller**:

PostController receives a POST request with a PostDTO.

2. **Service**: PostService.createPost() validates the content and tags.
3. **Repository**: PostRepository.save() persists the entity to H2/Oracle.
4. **Interconnectivity**: NotificationService is invoked to notify tagged users or existing connections.
5. **Response**: User is redirected back to the feed with a success flash attribute.

UI/UX & Frontend Logic

- **Server-Side Rendering**: Uses **Thymeleaf** for rapid and SEO-friendly rendering.
- **Dynamic Interactivity**: Subtle use of **JavaScript/AJAX** for actions like "Liking" or "Sharing" to provide a smooth UX without full page reloads.
- **Asset Pipeline**: Static assets are organized in /static/css and /static/js with modern standard compliance.

Technical Design Decisions

1. Choice of Thymeleaf over SPAs (React/Angular)

The decision to use **Thymeleaf** was driven by the need for a robust, server-side rendered application that integrates seamlessly with the Spring Boot ecosystem. This reduces the complexity of managing a separate frontend build pipeline and provides out-of-the-box SEO benefits and faster initial page loads.

2. Dual Database Strategy (H2 & Oracle)

By supporting both **H2** (for rapid local development) and **Oracle** (for enterprise-grade production), the project ensures developer productivity without sacrificing platform reliability. The JPA layer is carefully tuned to be database-agnostic through standard SQL and Hibernate dialects.

3. JWT and Session Hybrid Model

The application implements a unique hybrid security model. Standard web flows use session-based security for ease of use in browsers, while the REST API is secured with **stateless JWT tokens**. This allows the platform to support both a traditional web frontend and potential future mobile integrations simultaneously.

Future Roadmap

- **Cloud Storage**: Integration with AWS S3 or Azure Blob for high-performance image hosting.
- **Analytics Dashboard**: Enhanced visual metrics for Business and Creator accounts.
- **Mobile SDK**: Native Android/iOS connectivity via the expanded REST API.